



Caio Queiroz

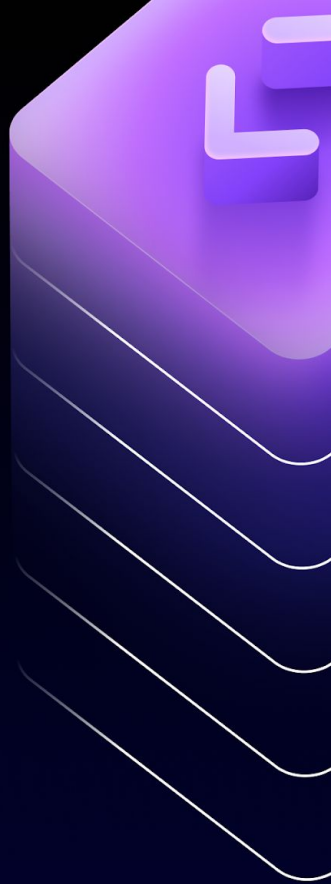
@caiocqueiroz

- +2 anos @ GitHub 🧡
- Antes em ThoughtWorks, SGS, IBM
- +20 anos de experiência em Tecnologia/Infraestrutura
- DevOps Background 🙌





Introdução ao Git





Agenda

Objetivos de aprendizado

- 01 O que é controle de versão?
- 02 Entender sistemas de controle de versões distribuídos, como o Git.
- 03 Criar um novo projeto Git e configure
- 04 Alterações no código usando Git
- 05 Praticar conceitos e comandos do dia a dia no Terminal e no IDE



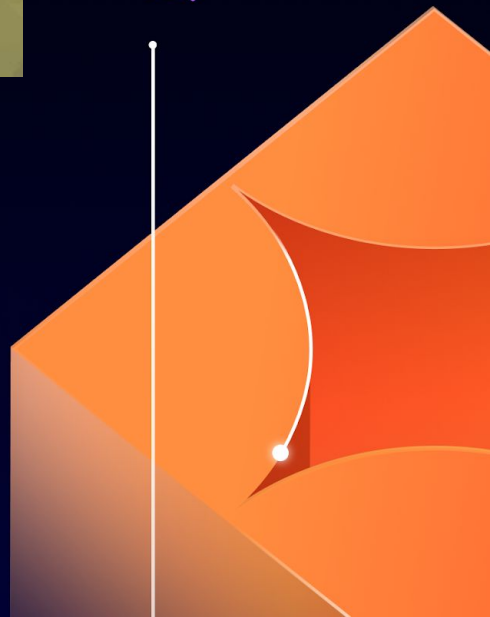


Version Control

No contexto enterprise, isso
Um Version Control System (VCS)
habilita:
permite:

- Governança de código
- Controle de histórico
- Compliance completo (audit trail)
- Segurança baseada em hash
- Rastreabilidade de histórico imutável
- Mudanças por commit
- Base para CI/CD e DevOps
- branches
- Recuperação determinística

Sem versionamento estruturado,
não existe engenharia em escala.



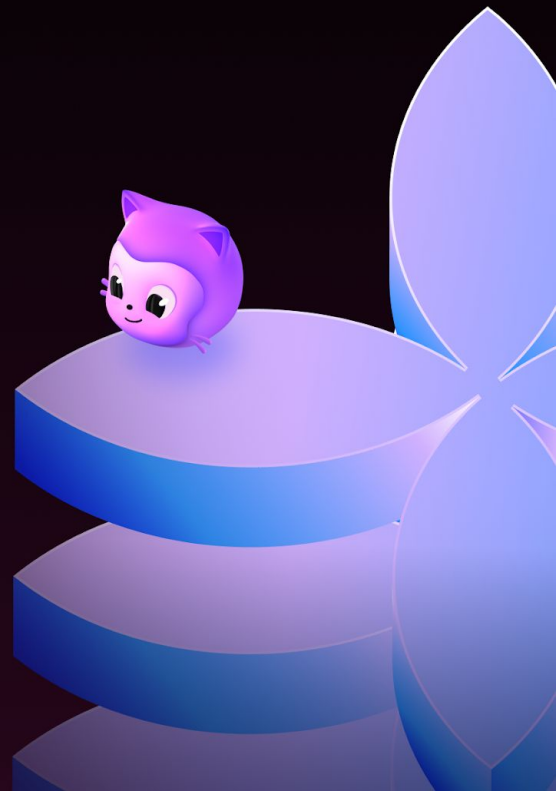


Git como Distributed Version Control System (DVCS)

- Comparado a Modelos centralizados (SVN, Perforce):

Principais características:

- Reduz latência operacional
- Cada clone contém o histórico completo (full commit graph)
- Escala para milhares de repositórios e desenvolvedores
- Operações locais (commit, branch, merge) independentes de rede
- Sincronização via remote (ex: origin)





Base dos workflows modernos

Conceitos principais (dia a dia)



Working tree

Define o estado atual do código baseado em um modelo de árvore.



Commit

Captura (snapshot) imutável com identificador único (hash)



Repositorio (.git)

Banco de objetos e metadados do git (definidos pelo arquivo .git)



Branch

Ponteiro móvel para uma sequência de commits



Base dos workflows modernos

Conceitos principais (dia a dia)



HEAD

Referencia para o commit atual. Aponta para onde você está na linha do tempo do projeto (working tree)



Tag

Marcação em um commit específico, não se move, é um ponto fixo na árvore.



Merge

Reconciliação de historicos, une as linhas do tempo que se criam durante o processo de desenvolvimento.



Remote

Referencia para repositório remoto (ex: github.com) . Permite sincronização dentro do sistema distribuído.

Push Envia.
Pull Traz.



Comandos Básicos Git

Operações Git usam comandos como :

- push
- pull

Ambos são sub comandos do comando git ex:

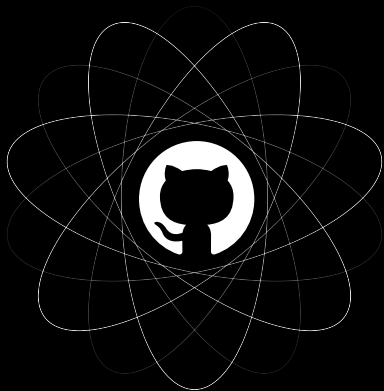
- git pull
- git push

Comandos são geralmente acompanhados por parâmetros (options) ex:

`git push --rebase`

<https://comandosgit.github.io/>





Git e GitHub qual a diferença

Git -> Controle de estado

GitHub -> Colaboração, Orquestração, Governança

Git (DVCS)

- Versionamento de código
- Estrutura de objetos (commits, tags, branches)
- Controle distribuído

GitHub (Plataforma)

- Camada de colaboração e governança
- Pull Request
- Regras de Proteção (Rulesets)
- CI/CD (Actions)
- Segurança associada ao código (GHAS)
- Governança de Projetos.
- Inteligencia Artificial (Copilot)
- Repositório Remoto



Como uma mudança realmente acontece

Modelo mental completo do fluxo Git

O ciclo de vida de uma mudança

1. Working tree - Você altera arquivos localmente
2. Staging area (index) - **git add** prepara alterações para o próximo commit
3. Commit - Snapshot imutável com identificador único (hash)
4. Branch - Ponteiro móvel que evolui com novos commits
5. Merge ou Rebase - Integração com outra linha de desenvolvimento
6. Remote (push) - Sincronização com o repositório remoto



Se você entende esse fluxo, os comandos deixam de ser memorização e passam a ser consequência lógica.



Componentes do GitHub Flow

Erros Comuns

- Commits grandes demais
- Mensagens de commit vagas (“update”, “fix”)
- Reescrever histórico compartilhado
- Force push em branch usada pelo time
- Versionar arquivos sensíveis (secrets)

Boas Praticas

- Commits pequenos e frequentes
- Mensagens claras e descritivas
- Branch por feature ou correção
- Pull Request para integração
- Histórico linear quando possível
- Nunca alterar histórico já compartilhado





DEMO



Git Foundations: o que aprendemos

Version control na prática

Conceito de Version Control System (VCS)

Histórico imutável, rastreabilidade e audit trail

Branching e merge como base de colaboração

Versionamento estruturado é o alicerce de colaboração, automação e segurança em escala.

Git – fundamentos técnicos

Working tree, repository, commit, branch, remote

Hash, objetos (tree, commit, tag)

Modelo distribuído (DVCS)

Erros comuns e melhores praticas

Git vs GitHub

Git = motor de versionamento

GitHub = plataforma de colaboração e governança

Operações Básicas

Configuração inicial

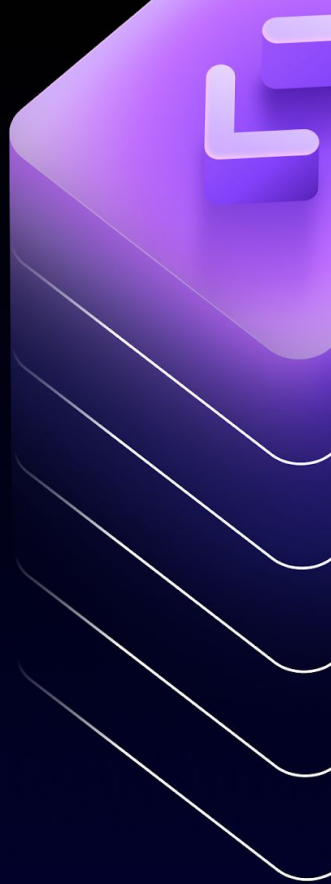
Comandos basicos e comuns no dia a dia

Prática no terminal e na IDE





Introdução ao GitHub





Agenda

Objetivos de aprendizado

- 01 Identificar e entender os fundamentos do GitHub.
- 02 Aprender sobre gestão de repositórios.
- 03 Entender o fluxo do GitHub (branches, commits, pull requests).
- 04 Explore as funcionalidades colaborativas do GitHub revisando questões e discussões.
- 05 Entender os diferentes recursos da GitHub
- 06 Como a plataforma integra os recursos .
- 07 Entender os diferentes tipos de conta dentro da GitHub.
- 08 Entender o que é o EMU (Enterprise Managed Users)



GitHub em 1 Frase

Plataforma onde os times constroem, colaboram e entregam software com mais velocidade e segurança

Código + Colaboração + Automação
Não é só repositório. É workflow de engenharia inteiro, do planejamento à entrega.
Fluxo padrão para sair de "mudança" -> "review"-> "merge" -> "entrega"



Product ▾ Solutions ▾ Resources ▾ Open Source ▾ Enterprise ▾ Pricing



Sign in

Sign up

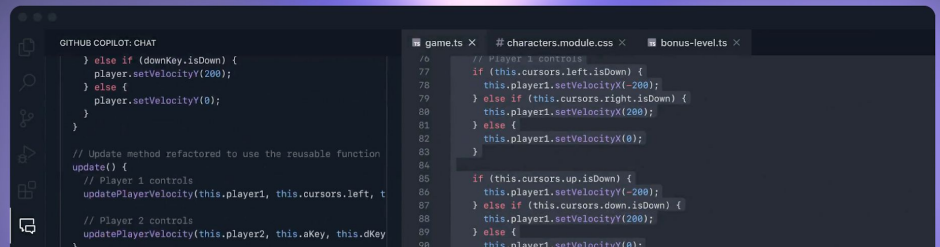
Build and ship software on a single, collaborative platform

Join the world's most widely adopted AI-powered developer platform.

Enter your email

Sign up for GitHub

Try GitHub Copilot



GitHub Platform

The AI Powered Developer
Platform to Build, Scale, and
Deliver Secure Software





A unidade básica de trabalho

Repositório (repo)

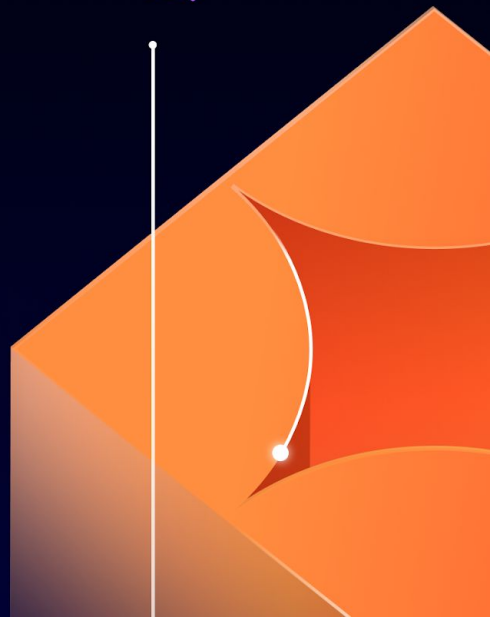
Repo = arquivos do projeto + históricos de revisões

O que o time precisa dominar

- Como criar e clonar repositórios, entender sua visibilidade (público, privado, interno)
- Como gerenciar os arquivos dentro do repositório (commits, merge, operações git)
- Commitar com mensagem boa (clareza e audit trail)
- Gerenciar branches e pull requests com um fluxo definido.

Uso real (enterprise)

- Repo bem estruturado = onboarding mais rápido, menos “tribal knowledge”, mais repetibilidade.





Componentes do GitHub Flow

Branches

Branches permitem desenvolver funcionalidades ou correções de forma isolada, sem impactar a branch principal.

Elas funcionam como um ambiente seguro para testar mudanças e corrigir erros

Commits

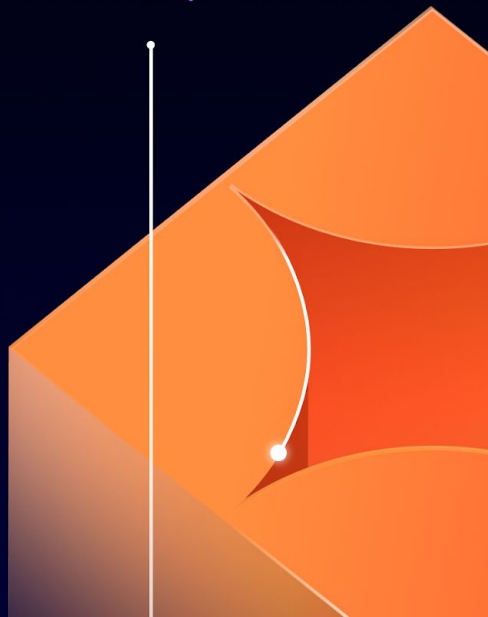
Commits representam registros de alterações feitas em um ou mais arquivos dentro de uma branch.

Cada commit possui um ID único, autor e data, garantindo rastreabilidade e histórico completo das mudanças no repositório. Eles funcionam como um “checkpoint” do código, permitindo auditoria e rollback quando necessário.

Pull Requests

Pull Requests são o mecanismo utilizado para propor a integração de alterações de uma branch para outra.

Elas permitem que revisores analisem o código, façam comentários e aprovem (ou solicitem ajustes) antes do merge, garantindo qualidade e colaboração.





GitHub Flow

O workflow padrão para entregar software com controle

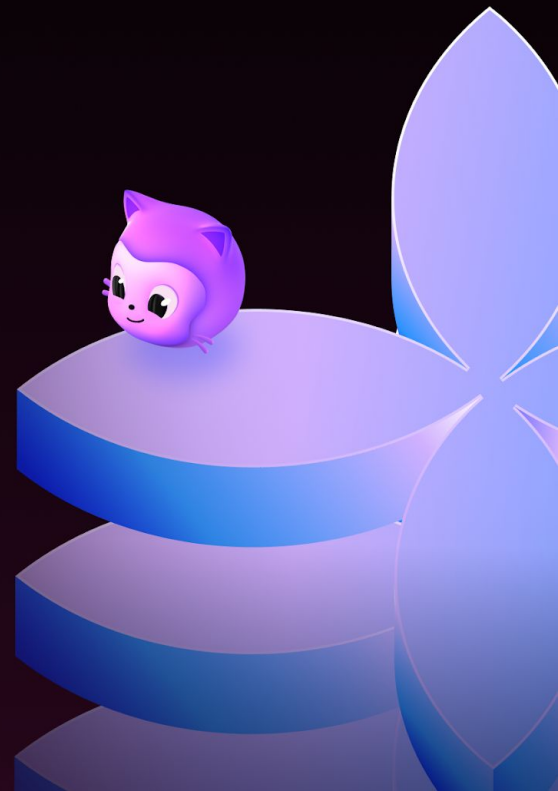
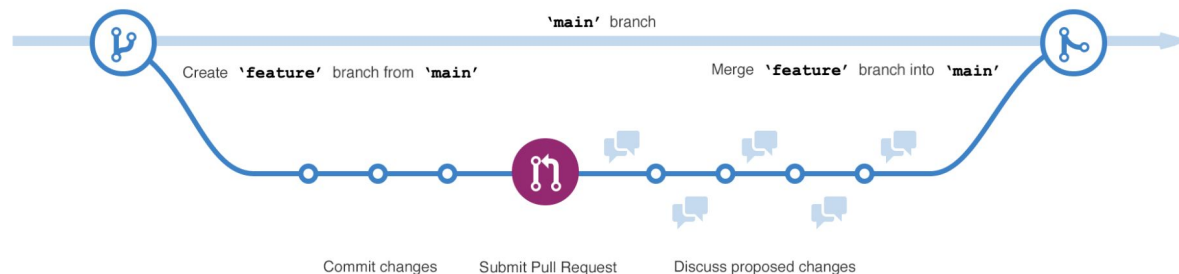
Um modelo simples e contínuo de desenvolvimento baseado em:

1. Branch a partir da main
2. Commits pequenos e frequentes
3. Pull Request (PR) para revisão
4. Checks automatizados (CI, testes, segurança)
5. Merge após aprovação
6. Deploy contínuo



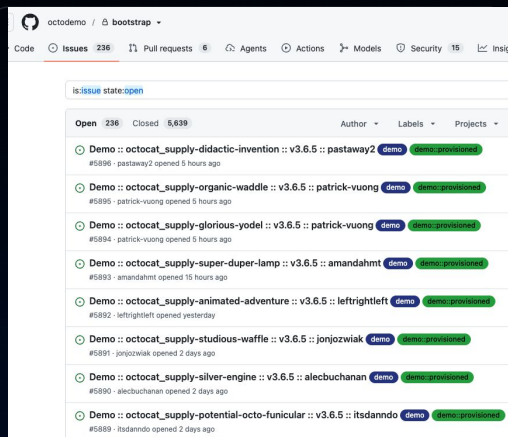
Fluxo resumido:

main → feature branch → PR → review + checks → merge → deploy



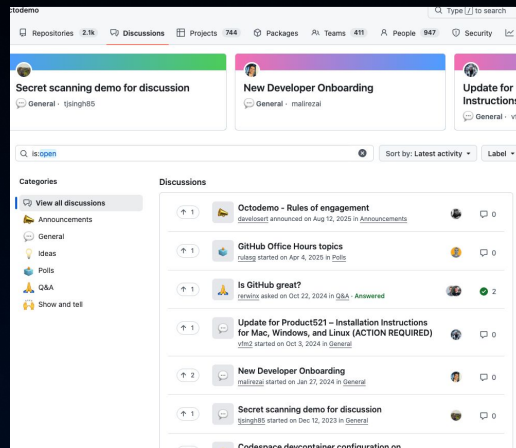
Onde o trabalho vive

Colaboração no GitHub



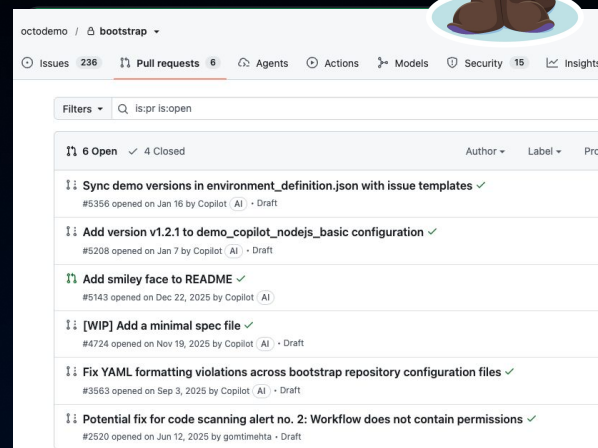
Issues

ideias, tarefas, bugs, tracking (com labels/milestones/projects).
Integração com Copilot Coding Agent.



Discussions

Q&A, anúncios, feedback – sem estar preso a código.



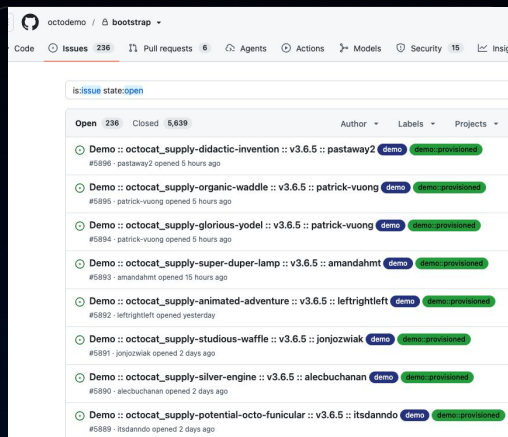
Pull Requests

Revisão + conversas técnicas + decisão antes de colocar sua mudança em produção.

..

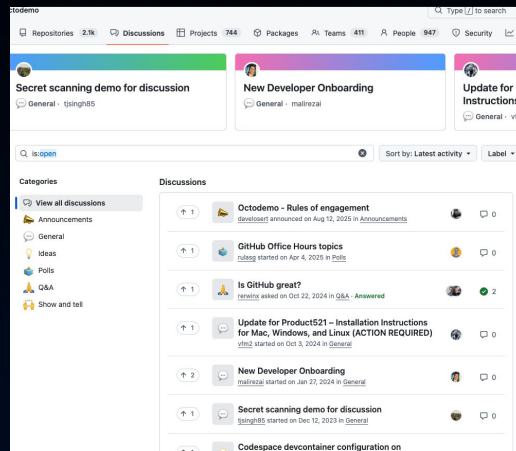
Desenvolver -> Revisar -> Automatizar -> Proteger -> Escalar

Produtos GitHub



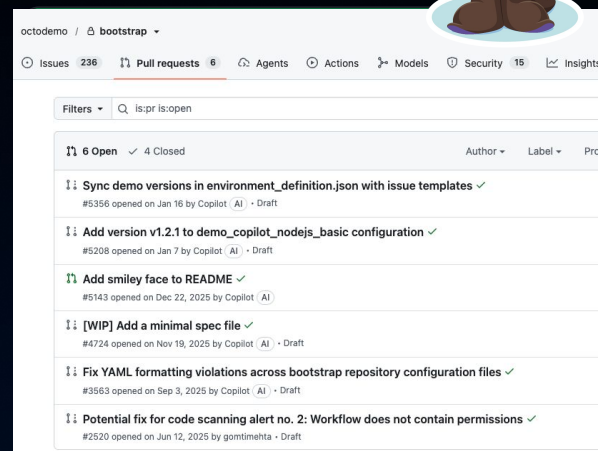
Repositorios

ideias, tarefas, bugs, tracking (com labels/milestones/projects).
Integração com Copilot Coding Agent.



Pull Requests

Q&A, anúncios, feedback – sem estar preso a código.

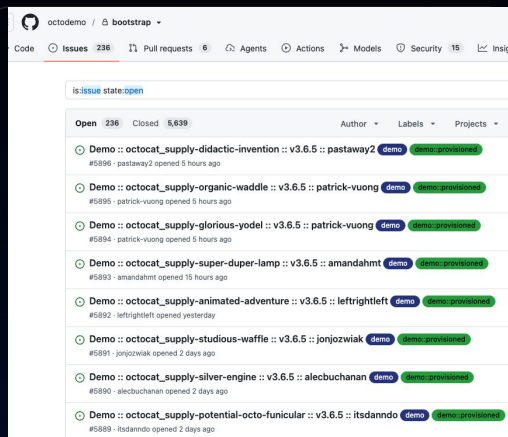


Actions

Revisão + conversas técnicas + decisão antes de colocar sua mudança em produção.
"

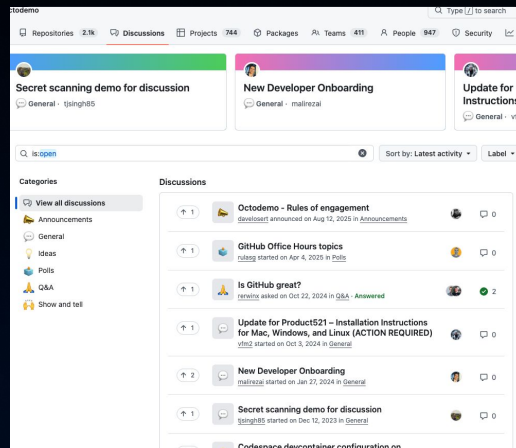
Desenvolver -> Revisar -> Automatizar -> Proteger -> Escalar

Produtos GitHub



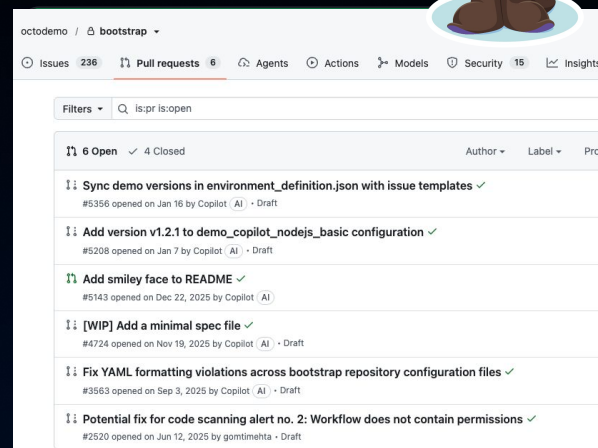
Copilot

ideias, tarefas, bugs, tracking (com labels/milestones/projects).
Integração com Copilot Coding Agent.



Advanced Security

Q&A, anúncios, feedback – sem estar preso a código.



Integração/Plataforma

Revisão + conversas técnicas + decisão antes de colocar sua mudança em produção.





Onde o valor acontece

Integração da plataforma GitHub

GitHub não é um conjunto de ferramentas soltas, é uma plataforma

Repositório

Base da plataforma.

- Armazena código, histórico e branches
- Dispara eventos (push, PR, merge)
- É o ponto central que conecta Actions, Copilot e Security





Onde o valor acontece

Integração da plataforma GitHub

GitHub não é um conjunto de ferramentas soltas, é uma plataforma

Copilot

Opera sobre o código do repositório.

- Sugere código, resolução de problemas no contexto do **repo/projeto/organização**
- Atua antes do commit e durante o review, revisa suas **pull requests** automaticamente (**CCR**).
- Crie ambientes e executa tarefas em "background" com apoio do **Actions (CCA)**
- Sugere e aplica soluções no seu código a partir do que foi encontrado pelo **Advanced Security (autofix)**





Onde o valor acontece

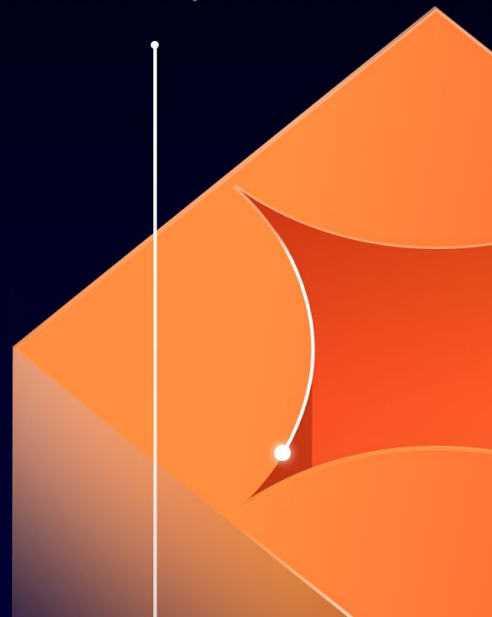
Integração da plataforma GitHub

GitHub não é um conjunto de ferramentas soltas, é uma plataforma

Pull Request

Camada de governança.

- **Copilot pode** gerar a descrição e resumo da sua pull request com um clique (baseado no conteúdo adicionado)
- Destino das sugestões geradas pelo **CCA** com revisão humano (**human in the loop**)
- Aciona automaticamente **Actions e Advanced Security (GHAS)**
- **Rulesets** para controle e "gates" baseados no seu fluxo e projeto





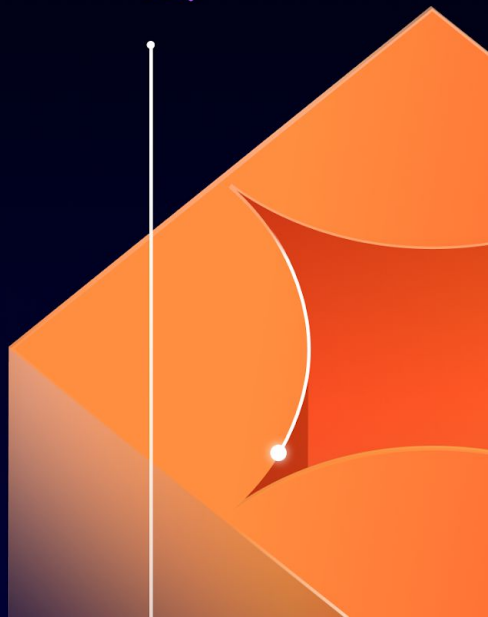
Onde o valor acontece

Integração da plataforma GitHub

GitHub não é um conjunto de ferramentas soltas, é uma plataforma

GitHub Actions

- Motor de automação orientado a eventos, pode ser acionado e acionar de diversas formas na **plataforma**
- Executa CI/CD ao abrir ou atualizar **Pull Requests**
- Fornece **status checks** obrigatórios para merge que se integram com **Rulesets e workflows**
- Ambiente onde o **CCA** executa suas tarefas para gerar **pull requests** com o código sugerido





Onde o valor acontece

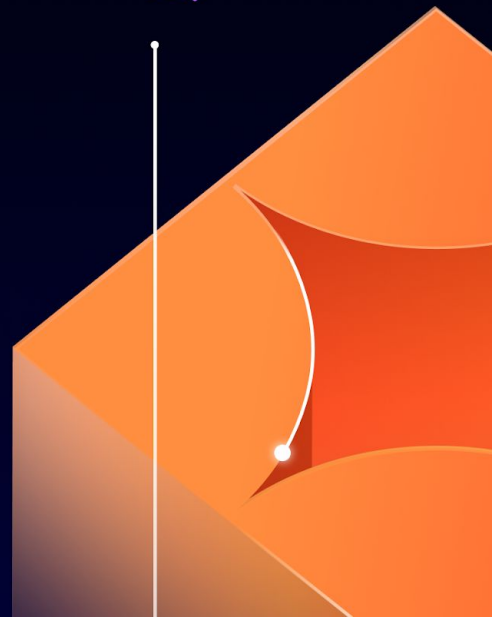
Integração da plataforma GitHub

GitHub não é um conjunto de ferramentas soltas, é uma plataforma

GitHub Advanced Security (GHAS)

Segurança integrada ao PR.

- Analise o código do **Repositório** utilizando Code scanning (CodeQL) e gera alertas com as descobertas
- Analise o código do **Repositório** e mostra secrets expostas, também evita que secrets cheguem ao **repositório remoto (push protection)**
- Analise as dependências do seu código.
- Bloqueia **merge** via required checks

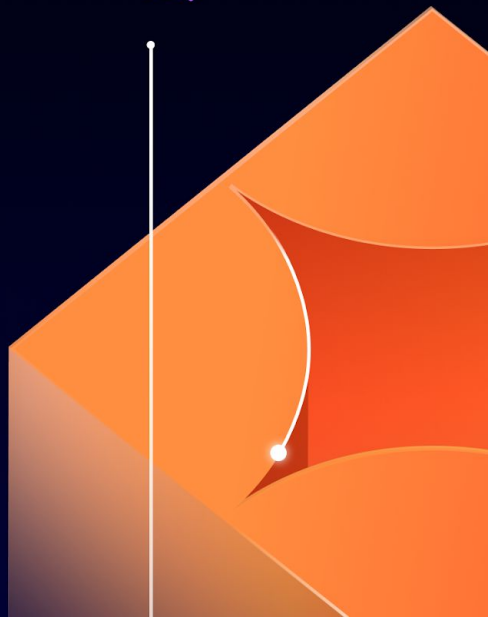




Extensões da plataforma além de código e CI/CD

Outros recursos que você deve conhecer

- **GitHub Packages** - Registry integrado ao repositório, Publicação de pacotes (npm, Maven, NuGet, Docker, etc.). Controle de acesso via permissões do repo, Integração direta com GitHub Actions.
- **GitHub Pages** - Hospedagem de sites estáticos direto do repositório. Publicação automática via branch ou Actions
- **Git LFS (Large File Storage)** - Gerenciamento de arquivos grandes. Armazena arquivos grandes fora do repositório principal
- **GitHub CLI (gh)** - Interface de linha de comando para GitHub. Criar PRs via terminal, Gerenciar Issues, Interagir com workflows.
- **Webhooks & API** - Integração com sistemas externos. Eventos de push, PR, release, Integração com ferramentas internas. Automação personalizada





Integração no Fluxo Real

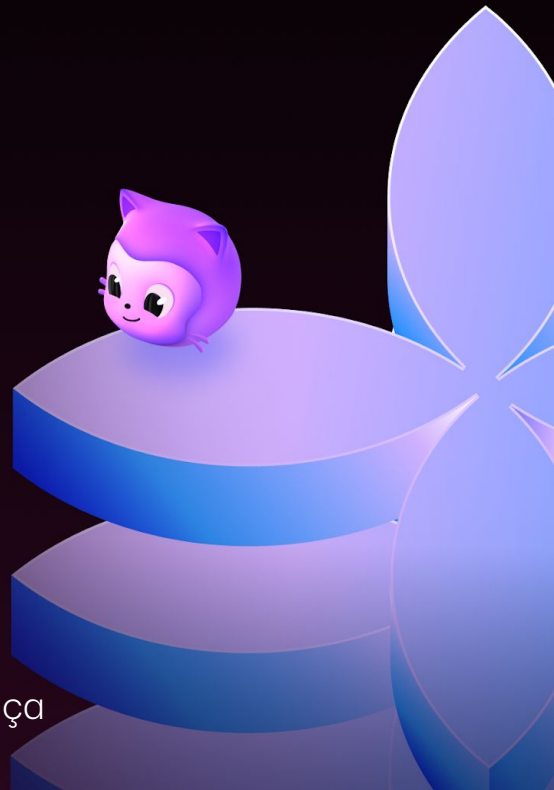


- → Copilot auxilia e acelera o desenvolvimento
- → PR centraliza e gerencia mudanças
- → Actions integra, implementa e valida
- → GHAS analisa e melhora segurança
- → Merge libera entrega

Porque isso importa?

- → Menos integrações externas frágeis
- → Menos contexto perdido
- → Segurança e qualidade embutidas no fluxo
- → Governança automática
- → Escalabilidade organizacional

O Valor está no fluxo contínuo onde produtividade, automação e segurança operam juntos.





Tipos de Planos - GitHub

GitHub Free (personal and organizations)

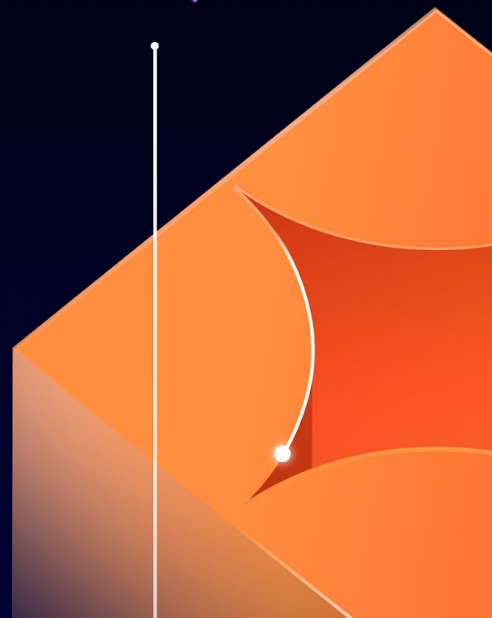
Inscrição no GitHub Free oferece a um novo usuário uma conta de usuário pessoal. Uma conta de usuário pessoal inclui repositórios públicos e privados ilimitados e colaboradores ilimitados.

GitHub Pro

GitHub Pro é semelhante ao GitHub Free, mas vem com recursos aprimorados. O plano é projetado para desenvolvedores individuais (usando sua conta pessoal) que desejam ferramentas e insights avançados em seus repositórios, mas não pertencem a uma equipe.

GitHub Team

O GitHub Team é um plano concebido para organizações, fornecendo recursos aprimorados de colaboração, segurança e gestão de fluxo de trabalho.





GitHub Enterprise

GitHub Enterprise oferece suporte aprimorado, segurança robusta, compliance e controles avançados para grandes organizações. garante políticas consistentes e centralizadas em toda a sua infraestrutura enterprise no GitHub. *Pay per use*

GitHub Enterprise Server

Versão "Self Hosted" da plataforma GitHub, permitindo total controle sobre a infraestrutura onde a plataforma esta sendo executada.

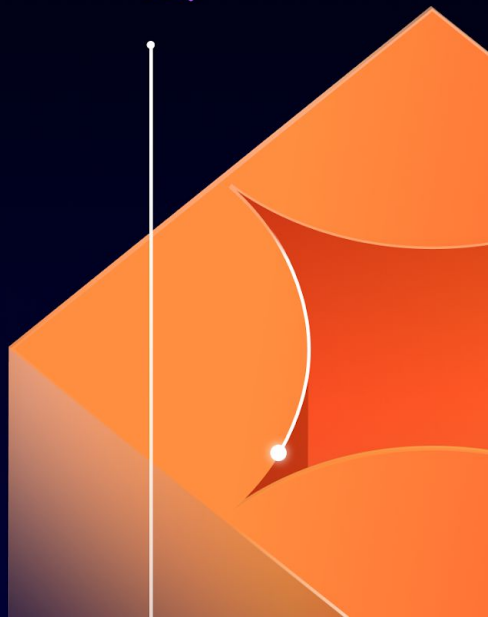
GitHub Enterprise Cloud

Essa é a versão em nuvem da plataforma, a versão cloud oferece mais flexibilidade e uma abordagem SaaS onde a GitHub cuida de toda a infraestrutura da plataforma, existem também funcionalidades extras que estão somente na versão em nuvem

Enterprise Managed Users (EMU)

Modelo corporativo de identidade onde os usuários provisionados via IdP (ex: Entra ID).

Sem contas pessoais independentes e com ciclo de vida controlado pela empresa





Tipos de conta - GitHub

Personal Account

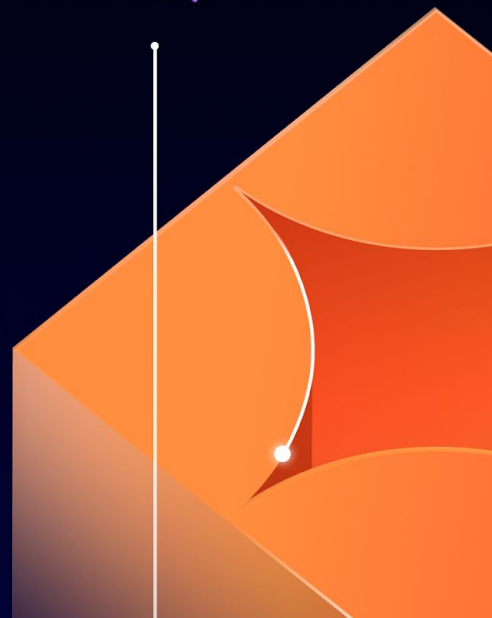
- identidade individual do usuário
- Executa ações (PRs, commits, reviews)
- Em ambiente EMU → identidade é provisionada e gerenciada pela empresa

Organization

- Unidade operacional onde vivem os repositórios
- Times, permissões e colaboração
- Pode haver múltiplas orgs sob o mesmo enterprise

Enterprise Account

- Camada superior de governança, políticas, billing e segurança
- Controla todas as organizations





DEMO



Introdução ao GitHub - O que aprendemos

O Que é GitHub

- Plataforma de colaboração baseada em Git
- Repositório como unidade de trabalho
- Código, histórico, PRs e Issues no mesmo lugar
- GitHub Flow
Branch → PR → review → merge → deploy
- Recursos Complementares
: Packages, Pages, LFS, CLI e APIS.

Produtos e Ecosistema

- Repositórios
Base da colaboração
- GitHub Actions
Automação e CI/CD integrada
- GitHub Copilot
Produtividade com IA
- GitHub Advanced Security
Segurança no fluxo (CodeQL, secrets, dependências)
- Integração da plataforma
Tudo conectado dentro do mesmo pipeline

Contas, Planos

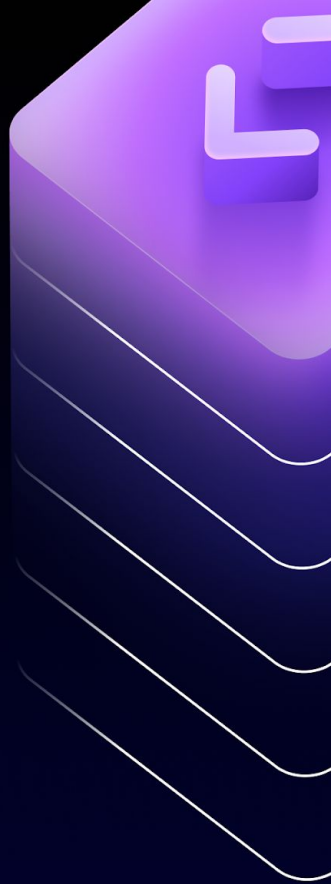
- Personal, Organization, Enterprise
- Enterprise Managed Users (EMU) Identidade controlada via IdP
- GitHub Enterprise Cloud
Governança, políticas e escala
- Uso consciente de recursos
Actions, Packages e consumo



GitHub conecta colaboração, automação, segurança e governança em uma única plataforma escalável.



Introdução ao Copilot





Efeito Copilot

- A ferramenta de IA para desenvolvedores mais adotada no mundo.

Aumente a felicidade de Dev

Aumente a produtividade do Dev

Acelere o desenvolvimento de software



Arquitetura

Porque usar o Copilot?

Porque é, também, a plataforma mais segura e aderente às exigências de conformidade do mercado.



LLM



Copilot Proxy

Context

Suggestions

```
runtime.go
1 package main
2
3 type Run struct {
4     Time int // in milliseconds
5     Results string
6     Failed bool
7 }
8
9 // Get average runtime of successful runs in seconds
10 func AverageRuntimeInSeconds(runs []Run) float64 {
11     var totalTime int
12     var failedRuns int
13     for _, run := range runs {
14         if run.Failed {
15             failedRuns++
16         } else {
17             totalTime += run.Time
18         }
19     }
20
21     averageTime := float64(totalTime) / float64(len(runs) - failedRuns) / 1000
22     return averageTime
23 }
```

Copilot



Code Completion

Receba **sugestões de código** contextualizadas ao seu projeto.

✓ Converta comentários em código

✓ Auto Preenchimento de código.

✓ Mostra alternativas



```
TS sentiment.ts > isPositive
1  #!/usr/bin/env ts-node
2
3  import { fetch } from "fetch-h2";
4
5  // Determine whether the sentiment of text is positive
6  // Use a web service
7  async function isPositive(text: string): Promise<boolean> {
8      const response = await fetch("http://text-processing.com/api/sentiment/", {
9          method: "POST",
10         body: `text=${text}`,
11         headers: {
12             "Content-Type": "application/x-www-form-urlencoded",
13         }
14     });
15     const json = await response.json();
16     return json.label == "pos";
17 }
```



Chat na IDE

Conversas no estilo Chat GPT com **contexto** do seu projeto.



Responde perguntas de código



Sugestão de correções de código.



Explica código



Aumenta legibilidade



Escreve testes unitarios



Traduz linguagens



Write unit tests for this function.



Workspace used /tests (rerun without)

Plan

1. Import necessary modules for testing.
2. Mock the `fetch` function to simulate API responses.
3. Write test cases to cover:
 - Positive sentiment response.
 - Negative sentiment response.
 - Neutral sentiment response.
 - Error handling.

Code

TS sentiment.test.ts



```
1+ import { isPositive } from './sentiment';
2+ import { fetch } from 'fetch-h2';
3+
4+ // sentiment.test.ts
5+
6+
7+ jest.mock('fetch-h2');
8+
9+ describe('isPositive', () => {
10+   it('should return true for positive sentiment', async () => {
11+     (fetch as jest.MockedFunction<typeof fetch>).mockResolvedValue({
```



Modo agente

Modo autônomo orientado a tarefas complexas

Planeja, executa e valida alterações de ponta a ponta

Interage com código, testes e terminal de forma contínua



✓ Ran command in terminal

The Spring Boot application has been built successfully. Now, let's run the application to expose the web UI for encoding and decoding text in Base64.

I'll start the Spring Boot application.

Run command in background terminal

```
mvn spring-boot:run
```

Run the Spring Boot application.

Continue

Cancel

Working Set (3 files)

Accept

Discard



📌 pom.xml

J Base64Controller.java src/main/java/com/example/helloworld

J HelloWorldApplication.java src/main/java/com/example/helloworld

+ Add Files...

Edit files in your workspace in agent mode (Experimental)



Agent ▾ GPT 4o ▾ ▶

Use o símbolo # para
usar uma variável do
chat. Com #file você
pode passar
conteúdo do arquivo
no espaço de
trabalho do Copilot

 lindseybocatto

`#file:py-coding.yml` review my code using coding standards defined in py-coding.yml

 GitHub Copilot

Used 2 references ▾

 prime.py:1-23 python
! py-coding.yml #file:py-coding.yml

Based on the `py-coding.yml` file you provided, here are some suggestions to improve your code:

1. Function names should be in `camelCase`. So, `CheckPrime` should be `checkPrime`.
2. Variable names should be in `snake_case`. So, `isPrime` should be `is_prime` and `userInput` should be `user_input`.
3. The docstring style should be `numpy`. So, the docstring for `checkPrime` should be updated.
4. Indentation should be 4 spaces, which your code already follows.

Here's your updated code:

```
def check_prime(number):  
    """  
    This function checks for prime numbers.  
  
    Parameters  
    -----  
    number : int  
        The number to check.  
  
    Returns  
    -----
```

```
4     if number == 2:  
5         isPrime = True  
6  
7     if number > 1:  
8         for i in range(2, number):  
9             if number % i == 0:  
10                isPrime = False  
11                break  
12            else:  
13                isPrime = True  
14  
15     if isPrime:  
16         print(number, "is prime")  
17     else:  
18         print(number, "is not prime")  
19  
20  
21 if __name__ == '__main__':  
22     userInput = int(input("Enter a number: "))  
23     CheckPrime(userInput)
```

PROBLEMS 13 OUTPUT DEB

@lindseybocatto → /workspaces/

Comandos /slash

/explain

Explica como o código selecionado funciona

/tests

Gera testes unitario para o código selecionado

/fix

Propõe uma solução para problemas no código selecionado

@workspace

Prefixe seus prompts para fazer uma pergunta sobre os arquivos do seu workspace

/help

Ajuda sobre o Copilot Chat

CHAT: GITHUB COPILOT



lindseybocatto

[/tests](#)



GitHub Copilot

Sure, here's an example set of unit tests for the BookSe

```
package com.github.demo.service;

import com.github.demo.model.Book;
import org.junit.After;
import org.junit.Before;
import org.junit.Test;

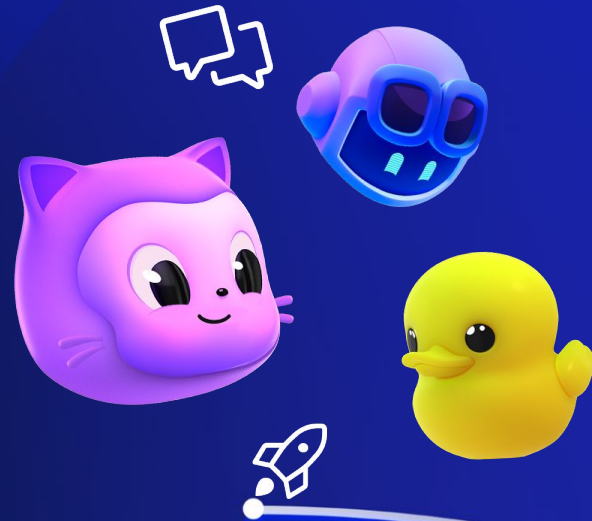
import java.util.List;

import static org.junit.Assert.assertEquals;
import static org.junit.Assert.assertNotEquals;

/**
 * Unit test for BookService
 */
public class BookServiceTest {

    private BookService bookService;
```

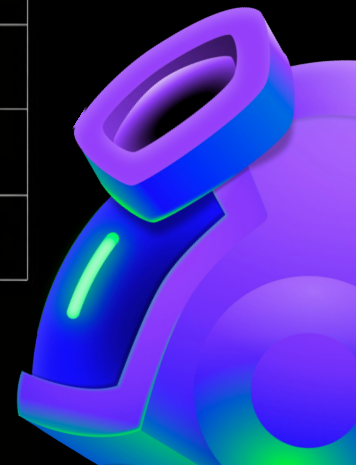
Unidades de Premium Request



Recursos do Copilot que usam PRUs

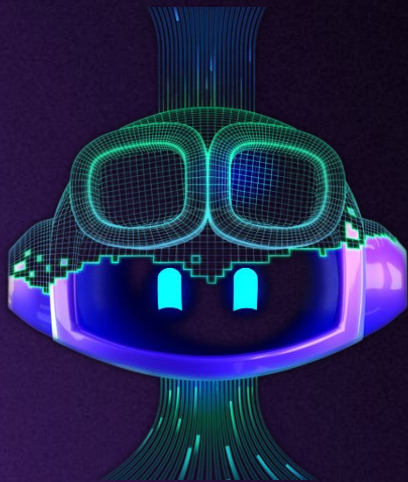


Copilot Chat	1 requisição premium por prompt do usuário, multiplicada pela taxa do modelo
Agente de Codificação do Copilot	1 requisição premium por sessão
Modo Agente no Copilot Chat	1 requisição premium por prompt do usuário, multiplicada pela taxa do modelo
Revisão de Código do Copilot	1 requisição premium é usada a cada vez que o Copilot posta comentarios no pull request.
Copilot Spaces	1 requisição premium por prompt do usuário, multiplicada pela taxa do modelo
Spark	4 requisições premium por prompt
Extensões do Copilot	1 requisição premium por prompt do usuário, multiplicada pela taxa do modelo





! Side notes !

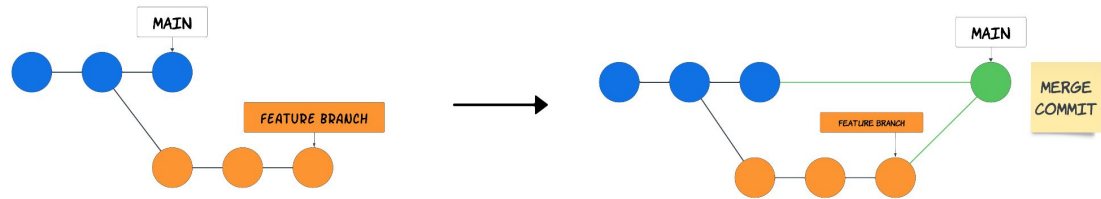


- Os desenvolvedores precisam realizar a devida diligência mesmo com a assistência do GitHub Copilot.
- Parte do código sugerido pode não ser adequado para o projeto.
- A ferramenta pode nem sempre compreender o contexto ou a intenção por trás do código.
- Os desenvolvedores devem revisar todo o código sugerido pelo GitHub Copilot.
- “**Copilot**” não é “Autopilot”

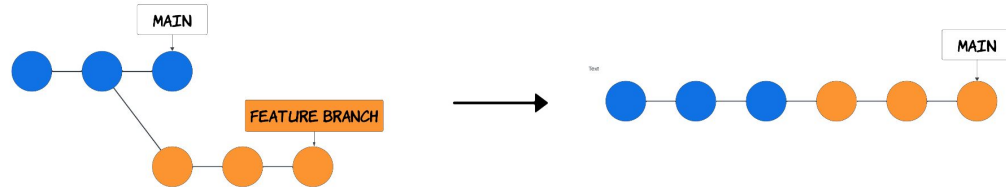


DEMO

MERGE



REBASE



SQUASH MERGE

